

EECE7352 Computer Architecture

Homework 3

Sakthivel P. Sivakumar

NUID: 002312294

Part E: (20 Points)

Consider the following RV32I RISC-V assembly code snippet:

```
# Loop with a loop-carried dependency
# x[i] = x[i-1] + x[i-2]

start:
    la t0, array           # Array base address
    lw t1, size            # Array size
    li t2, 2               # Start index

    # Initialize first two elements
    lw t3, 0(t0)           # Load x[0]
    lw t4, 4(t0)           # Load x[1]

reg_loop:
    bge t2, t1, reg_done

    add t5, t3, t4         # x[i] = x[i-1] + x[i-2]

    slli t6, t2, 2         # Calculate offset (i * 4)
    add t6, t0, t6         # Calculate address
    sw t5, 0(t6)           # Store x[i]

    # Shift values for next iteration
    mv t3, t4              # x[i-2] = x[i-1]
    mv t4, t5              # x[i-1] = x[i]

    addi t2, t2, 1         # i++
    j reg_loop

reg_done:
```

The code computes a Fibonacci sequence, where each element is the sum of the previous two elements (e.g., 1, 1, 2, 3, 5, 8, 13, 21, etc.). Using this code, produce RV32I RISC-V code for the following:

1. Provide the assembly code for an unrolled version of this code. Unroll the loop four times (there should be 4 iterations of the original loop in your code). Compute the improvement in performance for the unrolled loop versus the original loop (state any and all assumptions on RAW dependencies and control hazards).

Ans:

Performance Analysis:

1. Given Rolled Loop

For 1 iteration

- branch check (bge) - 1
- address calculation (slli, add) - 2
- computation (add) - 1
- store (sw) - 1
- value shifts (mv, mv) - 2
- increment (addi) - 1
- jump (j) - 1

The total number of instructions per iteration is **9**

2. Unrolled Loop

For 4 iterations

- Initial branch check - 1
- Address Calculation (once per 4 iterations) - 2
- Boundary checks - 3
- All the ALU operations (1 add + 1 store + 1 pointer update + 2 value shift + 1 increment) which totals to 6 instructions, since we are doing this 4 times, total = $6 \times 4 = 24$
- Jump - 1

Final total for 4 iterations is 31. Total instruction per iteration is 7.75

Speedup:

$$\text{Speedup} = 9/7.75 = 1.16 \text{ (i.e. 16\% improvement)}$$

Improvements:

- Address Calculation: Done once instead of 4 times, thus reducing 6 instructions
- Loop Overhead: Reduced from 4 to 1 jump (saving 3 instructions)

Assumptions:

The results I got is based on the following assumptions that a branch misprediction incurs a 2-cycle penalty, and each Read-After-Write (RAW) data dependency results in a 1-cycle stall. The key dependencies considered are from the 'add t5' instruction to both the 'sw t5' and 'mv t4, t5' instructions.

For the **original loop**, the base execution time is 9 cycles per iteration. Additionally, it also incurs approximately 2 cycles of stalls due to data dependencies, assuming a 75% branch prediction

accuracy, the avg. branch penalty is another 0.5 cycles. Summing all this we get approximately 11 – 12 cycles per iteration.

For the **unrolled loop**, a set of 4 iterations is estimated to take around 40 to 44 cycles. This version benefits from reduced branch penalties, as it executes 3 fewer branches for every 4 iterations. Although the same RAW dependencies exist for each individual computation, the reduced branch overhead lowers the effective average time to approximately 10 to 11 cycles per iteration.

2. Provide the assembly code for a software-pipelined version of this code. Compute the improvements in performance for this new version of the code (state any and all assumptions on RAW dependencies and control hazards).

Ans:

In this part I have implemented a software pipelined version of the given Fibonacci sequence calculation code.

The Key optimizations applied are:

- Loop unrolling - processes 4 elements per loop iteration
- Software pipelining – overlaps computation and storage from different logical iterations
- Offset based addressing – single address calculation + offsets (4, 8, 12 etc..)
- Early computation: calculating $x[i+4]$ at the end of iteration for next loop

```
19  reg_loop_swp:
20      # Iteration 1 (store prev, compute next)
21      sw      t5, 0(t6)    # Store x[i]
22      mv      t3, t4
23      mv      t4, t5
24      add     t5, t3, t4    # Compute x[i+1]
25
26      # Iteration 2
27      sw      t5, 4(t6)
28      mv      t3, t4
29      mv      t4, t5
30      add     t5, t3, t4
31
32      # Iteration 3
33      sw      t5, 8(t6)
34      mv      t3, t4
35      mv      t4, t5
36      add     t5, t3, t4
37
38      # Iteration 4
39      sw      t5, 12(t6)
40      mv      t3, t4
41      mv      t4, t5
42      add     t5, t3, t4    # Compute x[i+4] for next iteration
```

Figure: All 4 iterations of the software pipelined version code (unrolled loop)

Instruction count per 4 iterations

- Prologue instructions – 5
- Unrolled iterations – 4 (4 instructions per iteration, so for 4 iterations) – $4 \times 4 = 16$
- Each iteration (sw + mv + mv + add)
- Loop overhead – 3

Total for 4 iterations (16+3) = 19 instructions

For 1 iteration = 4.75 instructions

Speedup:

Speedup = $9/4.75 = 1.89$ (i.e. ~ 90% improvement)

Improvements:

- 47% less instruction compared to the original code
- 90% improvement in the speedup

Assumptions:

Taking the same 5 stage pipeline example, branch prediction is 90% accurate and a misprediction incurs a 2-cycle penalty and RAW dependencies are eliminated by the software pipelining.

For the **original loop**: It consists of 9 instructions. It incurs 2 RAW stalls and a 0.2 cycle branch penalty per iteration, resulting in an estimated total of ~11.2 cycles per iteration.

For **SW-pipelining loop**: The average instructions per iteration is 4.75 and incurs 0 RAW stalls. The branch penalty is reduced to 0.05 cycles per iteration (since the branch occurs once per four iterations). This results in an estimated total of ~4.8 cycles per iteration.

3. The Fibonacci number sequence appears in the natural world. Describe two examples where we see this sequence appear in nature.

Ans:

The Fibonacci sequence appears frequently in the natural world, especially in biology

Example 1: Flower Petals

The no. of petals on a flower is a very common example of fibonacci series. This pattern emerges from the way the plant's leaves, petals, and other structures are formed at the growing tip.

- 3 Petals – lily
- 5 Petals – Hibiscus
- 8 petals – Delphinium
- 13 petals – Marigold
- 21 petals – Astor

Example 2: Seed Heads

A more intricate and fascinating case is the **head of a sunflower**. If we observe the seeds, they form two distinct sets of spirals, one arcing clockwise and the other counterclockwise. The number of spirals in each direction is almost always a pair of sequential Fibonacci numbers, such as 34 in one direction and 55 in the other. This efficient biological arrangement has a specific name: **phyllotaxis**.